

XIAMEN UNIVERSITY MALAYSIA
厦門大學馬來西亞分校

单调栈：寻找下一个更大或更小元素

Monotonic Stack

作者：Wu Youfan
School of Computing and Data Science, Xiamen University Malaysia

2026 年 11 月 2 日

目录

XIAMEN UNIVERSITY MALAYSIA
厦門大學馬來西亞分校

1. 什么是单调栈
2. 单调栈的原理与代码
3. 经典例题
4. 总结



目录

1 什么是单调栈

2 单调栈的原理与代码

3 经典例题

4 总结

单调栈的直观理解

单调栈：栈内元素始终保持单调递增或单调递减的栈。

生活中的类比：排队看身高

- 一群人排队，你想知道每个人**右边第一个比他高的人**是谁。
- 如果来了一个很高的人，那么排在他前面、比他矮的人，就**永远被挡住了**。
- 那些“又矮又被挡住”的人，可以直接踢出队列，不用再管了。
- 剩下的队伍，从左到右一定是一个比一个高（单调递增）。

遇到更强的，就把前面不如自己的全干掉，栈里永远保持有序

单调栈能解决什么问题

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

核心题型：寻找每个元素的**下一个更大元素**或**下一个更小元素**。

四类常见变体

- 找右边第一个比自己大的数（单调递减栈）。
- 找右边第一个比自己小的数（单调递增栈）。
- 找左右两边第一个比自己小的数（柱状图最大矩形）。
- 找左边第一个比自己大的数（循环数组下一个更大元素）。

暴力解法的痛点

- 对每个元素都往后遍历找答案，最坏时间复杂度 $O(n^2)$。
- 单调栈能让每个元素最多进栈一次、出栈一次，优化到 $O(n)$。

单调栈的本质：把那些已经不可能成为答案的元素提前淘汰

目录

1 什么是单调栈

2 单调栈的原理与代码

3 经典例题

4 总结

原理：如何维护单调性

以“寻找下一个更大元素”为例：目标是从左往右，找每个人右边第一个比自己高的。

维护单调递减栈

- 栈内存放的是元素的下标（或位置）。
- 遍历当前元素 $a[i]$ 时，比较栈顶元素。
- 如果 $a[i]$ **大于**栈顶元素，说明栈顶元素找到了它的下一个更大元素。
- 此时栈顶元素出栈，并记录它的答案为 $a[i]$。
- 不断重复，直到栈空或 $a[i]$ 小于等于栈顶，再把当前元素入栈。

入栈前，先弹出所有比自己小或等于的元素

单调栈的通用代码框架

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```cpp
vector<int> nextGreater(vector<int>& a) {
    int n = a.size();
    vector<int> ans(n, -1);  // -1 表示找不到
    stack<int> st;

    for (int i = 0; i < n; i++) {
        // 把破坏单调性的元素弹出去 *
        while (!st.empty() && a[st.top()] < a[i]) {
            ans[st.top()] = a[i];
            st.pop();
        }
        st.push(i);           // 当前元素入栈
    }
    return ans;
}
```

核心就一个 while：弹出、记录答案、入栈

目录

1 什么是单调栈

2 单调栈的原理与代码

3 经典例题

4 总结

例题一：下一个更大元素

题目：给定两个没有重复元素的数组 `nums1` 和 `nums2`，其中 `nums1` 是 `nums2` 的子集。请你找出 `nums1` 中每个元素在 `nums2` 中的下一个比其大的值。

示例

- 输入：`nums1 = [4, 1, 2], nums2 = [1, 3, 4, 2]`
- 输出：`[-1, 3, -1]`
- 解释：4 在 nums2 中找不到更大的；1 右边第一个更大的是 3；2 找不到。

下一个更大元素：解法

核心思路：先对 `nums2` 使用单调栈预处理出所有元素的下一个更大元素，存入哈希表，再遍历 `nums1` 查表。

关键步骤

- 第一步：遍历 `nums2`，维护单调递减栈。
- 第二步：如果 `nums2[i]` 大于栈顶元素，弹出并记录结果。
- 第三步：把所有记录存进 `map`。
- 第四步：遍历 `nums1`，去 `map` 里取答案。

本质是对 nums2 做一遍单调栈，用 map 映射答案

下一个更大元素：代码


```cpp
vector<int> nextGreaterElement(vector<int>& nums1, vector<int>& nums2) {
    unordered_map<int, int> mp;
    stack<int> st;

    for (int x : nums2) {
        // 栈顶比当前小，说明找到了答案 *
        while (!st.empty() && st.top() < x) {
            mp[st.top()] = x;
            st.pop();
        }
        st.push(x);
    }

    vector<int> ans;
    for (int x : nums1) {
        if (mp.count(x)) ans.push_back(mp[x]);
        else ans.push_back(-1);
    }
```

下一个更大元素：代码


```
19      return ans;
20  }
```

例题二：每日温度

题目：给定一个整数数组 `temperatures`，表示每天的温度。请计算每一天需要等待几天才会出现更高的温度。如果之后都不会升高，请在该位置用 0 来代替。

示例

- 输入：`temperatures = [73, 74, 75, 71, 69, 72, 76, 73]`
- 输出：`[1, 1, 4, 2, 1, 1, 0, 0]`
- 解释：第一天 73，第二天 74 比它高，所以等 1 天。

每日温度：解法

核心思路：求每个元素右边第一个比自己大的元素的下标差值。由于求的是距离，栈内必须存下标而不是数值。

关键步骤

- 第一步：建立一个栈 `st` 存下标，答案数组 `ans` 初始为 0。
- 第二步：遍历当前温度的下标 `i`。
- 第三步：当栈不空且当前温度大于栈顶温度时，弹出栈顶 `j`，令 `ans[j] = i - j`。
- 第四步：把当前下标 `i` 入栈。

存下标比存数值更好用，既能比较，又能计算距离

每日温度：代码


```cpp
vector<int> dailyTemperatures(vector<int>& T) {
    int n = T.size();
    vector<int> ans(n, 0);
    stack<int> st;   // 存的是下标

    for (int i = 0; i < n; i++) {
        // 当前温度比栈顶高，栈顶找到答案 *
        while (!st.empty() && T[i] > T[st.top()]) {
            int j = st.top();
            st.pop();
            ans[j] = i - j;  // 计算等待天数
        }
        st.push(i);
    }
    return ans;
}
```

例题三：柱状图中最大的矩形

题目：给定 n 个非负整数，用来表示柱状图中各个柱子的高度。每个柱子彼此相邻，且宽度为 1。求在该柱状图中，能够勾勒出来的矩形的最大面积。

示例

- 输入：`heights = [2, 1, 5, 6, 2, 3]`
- 输出：10
- 解释：最大矩形由高度为 5 和 6 的柱子拼成，宽为 2，面积为 10。

柱状图中最大的矩形：解法

核心思路：对于每根柱子，找到它左边和右边第一个比它矮的柱子，就能算出以它为高的最大矩形面积。

关键步骤

- 第一步：在数组两端各加一个高度为 0 的哨兵，避免处理空栈边界。
- 第二步：维护一个单调递增栈，存下标。
- 第三步：当遇到高度小于栈顶的柱子时，说明栈顶柱子的右边界确定了。
- 第四步：弹出栈顶，计算面积。左边界就是弹出后的新栈顶，右边界是当前下标。

加上哨兵，让代码更好写，省去特判

柱状图中最大的矩形：代码


```cpp
int largestRectangleArea(vector<int>& h) {
    h.insert(h.begin(), 0);  // 头部加哨兵
    h.push_back(0);          // 尾部加哨兵
    stack<int> st;
    int ans = 0;

    for (int i = 0; i < h.size(); i++) {
        // 遇到比栈顶矮的，说明栈顶的右边界到了 *
        while (!st.empty() && h[st.top()] > h[i]) {
            int height = h[st.top()];
            st.pop();
            int width = i - st.top() - 1; // 宽度 = 右 - 左 - 1
            ans = max(ans, height * width);
        }
        st.push(i);
    }
    return ans;
}
```

目录

1 什么是单调栈

2 单调栈的原理与代码

3 经典例题

4 总结



总结：单调栈使用指南

题型	栈的单调性	触发弹栈的条件
下一个更大元素	单调递减	当前元素 $>$ 栈顶元素
下一个更小元素	单调递增	当前元素 $<$ 栈顶元素
接雨水 / 柱形图	单调递增	当前元素 $<$ 栈顶元素

一句话总结

- 求“下一个更大”，就维护“递减栈”，来了“更大”的就弹。
- 求“下一个更小”，就维护“递增栈”，来了“更小”的就弹。

存下标不仅能比较，还能直接算距离，比存数值更实用

总结：常见易错点

- **栈里存下标还是数值**：求值存数值，求距离（每日温度、宽度）存下标。
- **等于号的处理**：弹栈条件里到底带不带等号？如果求严格更大，就不带；如果求大于等于，就带等号。
- **栈空越界**：弹栈时一定要先判断栈非空，即 `while(st.empty())`！必须在前面，顺序不能反。
- **哨兵技巧**：在数组首尾添加 0，省去大量 while 结束后清理栈的代码，极大地简化了边界判断。

单调栈写错，往往是等号没想清，或者忘了判空

附录：单调栈标准模板

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```cpp
// 通用找左边/右边第一个更大/更小元素
// type = 0: 右边第一个严格更大
// type = 1: 右边第一个严格更小
vector<int> nextGreaterOrSmaller(vector<int>& a, int type) {
    int n = a.size();
    vector<int> ans(n, -1);  // 默认找不到为 -1
    stack<int> st;           // 存下标

    for (int i = 0; i < n; i++) {
        // 如果是要找更大，遇到比栈顶大的就弹
        // 如果是要找更小，遇到比栈顶小的就弹
        while (!st.empty() &&
               ((type == 0 && a[i] > a[st.top()]) ||
                (type == 1 && a[i] < a[st.top()]))) {
            ans[st.top()] = i;  // 记录下标
            st.pop();
        }
        st.push(i);
```

附录：单调栈标准模板


```
19      }
20      return ans;
21  }
```